



Bagify — Code Review & Viva Preparation Guide

A microservices-based e-commerce platform built with **Spring Boot** (backend) and **React + TypeScript** (frontend).

Table of Contents

1. [System Architecture & Design](#)
2. [Data Model & Database Strategy](#)
3. [Security & Authentication](#)
4. [Order Lifecycle & Business Logic](#)
5. [Concurrency & Transaction Management](#)
6. [Payment Gateway Integration \(PayHere\)](#)
7. [Inter-Service Communication](#)
8. [Notification Service](#)
9. [Frontend Architecture](#)
10. [Testing Strategy](#)
11. [Potential Weaknesses & Improvements](#)

1. System Architecture & Design

Q1: Why did you choose microservices over a monolith?

Answer:

The application is split into `user`, `product`, `order`, `notification`, and `api-gateway` services because:

- **Fault isolation** — If the `order` service fails, buyers can still browse products and log in.
- **Independent scaling** — The read-heavy `product` service can be scaled separately from the write-heavy `order` service.
- **Domain separation** — Each team/developer can work on one bounded context without breaking others.

Trade-off: Distributed systems add complexity — inter-service calls can fail, data consistency across services must be managed, and local development requires running 5+ services.

Q2: What is the role of the Eureka Server?

Answer:

Eureka is a **Service Registry**. Instead of hardcoding `http://localhost:8082` in the `order` service to call `product`, each service registers itself by name on startup:

```
spring:
  application:
    name: product # registered name
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
```

Other services discover it by logical name: `lb://product`. The `lb://` prefix enables **client-side load balancing** across multiple instances automatically.

Q3: What does the API Gateway do? Why is it important?

Answer:

The API Gateway (port `8080`) is the **single entry point** for the React frontend. It:

1. **Routes** requests to the right service based on the path prefix:

```
routes:
- id: product-service
  uri: lb://product
  predicates:
    - Path=/api/v1/products/**, /api/v1/reviews/**, /api/v1/complaints/**
- id: order-service
  uri: lb://order
  predicates:
    - Path=/api/v1/orders/**
```

1. **Centralises CORS** — rather than configuring CORS in every service, it is defined once in the gateway with `allowedOrigins: http://localhost:5173`.
 2. **Hides internal topology** — the `notification` service's `/internal/**` routes are intentionally **not listed** in the gateway, making them unreachable from the internet.
-

Q4: What is the startup order of your services and why?

Answer:

1. eureka-server ← must be first; all others register here
2. user ← auth; gateway depends on JWT validation logic
3. product ← no dependencies
4. notification ← no dependencies
5. order ← depends on product (stock check)
6. api-gateway ← last; routes to all others after they are up

Starting the gateway before Eureka finishes registering services causes routing failures.

2. Data Model & Database Strategy

Q5: Why does each service have its own database?

Answer:

This is the **Database-per-Service** pattern. Each service owns its private schema:

Service	Schema	Key Tables
user	BagifyUser	users, buyers, sellers
product	BagifyProducts	products, reviews
order	BagifyOrders	orders, order_items
notification	stateless	—

Why: Sharing a database creates tight coupling — a schema change in `product` would break the `order` service. Separate schemas let each service evolve independently.

Q6: How do you query data across services without shared databases? (Denormalization)

Answer:

Cross-service SQL joins are impossible. We use **denormalization** — capturing a snapshot of external data at write-time.

When an order is placed, `OrderItem` stores a **snapshot** of the product:

```
// OrderItem.java
private String productName; // captured from product service
private String imageUrl;    // captured from product service
private Double priceAtPurchase; // price at time of purchase, NOT current price
private String sellerId;
```

This ensures order history remains accurate even if the seller later changes the product name, image, or price.

Q7: Explain the JPA inheritance strategy used for the User entity.

Answer:

We use `InheritanceType.JOINED`:

```
@Entity
@Table(name = "users")
@Inheritance(strategy = InheritanceType.JOINED)
public class User { ... }

@Entity
@Table(name = "sellers")
@PrimaryKeyJoinColumn(name = "id")
public class Seller extends User {
    private String businessName;
    private String registrationNumber;
    private Seller.VerificationStatus verificationStatus;
    ...
}
```

- `users` table holds common fields (name, email, password, role).
- `sellers` table holds seller-specific fields joined on `id`.
- This avoids a bloated single table with many nullable columns.

Q8: Why is `@JsonIgnore` used on the password and reset token fields?

Answer:

To prevent sensitive fields from being serialised into any API response:

```
@Column(nullable = false)
@JsonIgnore
private String password;

@Column
@JsonIgnore
private String passwordResetToken;

@Column
@JsonIgnore
private LocalDateTime passwordResetTokenExpiry;
```

Without `@JsonIgnore`, these fields would appear in any endpoint that returns a `User` object, which is a critical **data leakage** vulnerability.

3. Security & Authentication

Q9: How does JWT authentication work across your microservices?

Answer:

Step 1 — Login (user service): On valid credentials, `JwtUtil.generateToken(user)` creates a signed JWT containing `userId`, `email`, and `role`. A separate `refreshToken` with a longer expiry is also issued.

Step 2 — Request (any service): The frontend attaches the token to every API call:

```
Authorization: Bearer <token>
```

Step 3 — Validation (JwtFilter in each service):

```
String jwt = authHeader.substring(7); // strip "Bearer "  
String email = jwtUtil.extractEmail(jwt);  
Integer userId = jwtUtil.extractUserId(jwt);  
String role = jwtUtil.extractRole(jwt);  
  
// Build Spring Security context  
UsernamePasswordAuthenticationToken authToken =  
    new UsernamePasswordAuthenticationToken(email, null,  
        List.of(new SimpleGrantedAuthority("ROLE_" + role)));  
authToken.setDetails(userId); // userId accessible in controllers via  
authentication.getDetails()  
SecurityContextHolder.getContext().setAuthentication(authToken);
```

Q10: What happens when a JWT token expires? How does the frontend handle it?

Answer:

When a JWT expires, the `JwtFilter` catches the exception and immediately returns **401**

Unauthorized:

```
} catch (Exception e) {  
    SecurityContextHolder.clearContext();  
    response.setStatus(HttpStatus.UNAUTHORIZED);  
    response.getWriter().write("{\"error\": \"Token expired or invalid\"}");  
    return; // stop filter chain – do not proceed to controller  
}
```

Key fix noted in code comments: Previously the filter silently cleared the context and fell through, causing Spring Security to return **403 Forbidden** instead of 401. The frontend's token

refresh logic only triggers on **401**, so tokens were never refreshed and users were wrongly logged out. Returning 401 explicitly from the filter fixes this.

Q11: How is Role-Based Access Control (RBAC) enforced?

Answer:

`@EnableMethodSecurity` is declared in `SecurityConfig`, enabling `@PreAuthorize` on controller methods:

```
// Only buyers can check out
@PostMapping("/checkout")
@PreAuthorize("hasRole('BUYER')")
public ResponseEntity<OrderResponse> checkout(...) { ... }

// Only sellers can update their item's fulfillment status
@PutMapping("/{orderId}/items/{itemId}/status")
@PreAuthorize("hasRole('SELLER')")
public ResponseEntity<OrderResponse> updateItemStatus(...) { ... }

// Only admins can mark items as DELIVERED
@PutMapping("/{orderId}/items/{itemId}/status/admin")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<OrderResponse> updateItemStatusAdmin(...) { ... }
```

An additional **runtime check** is done for sellers — even after the `@PreAuthorize`, the service verifies the seller actually owns the item:

```
if (!String.valueOf(sellerId).equals(item.getSellerId())) {
    throw new ResponseStatusException(HttpStatus.FORBIDDEN, "You do not own this item");
}
```

Q12: How is the session managed? Is it stateful or stateless?

Answer:

Completely **stateless** — no HTTP sessions are created on the server:

```
.sessionManagement(session ->
    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
)
```

Every request must carry a valid JWT. This is essential for microservices — there is no shared session store between the `user` and `order` services.

Q13: How does password reset work securely?**Answer:**

1. User submits their email to `/api/v1/auth/forgot-password`.
2. A random UUID token is generated and stored in the `User` entity with a **15-minute expiry**:

```
java String token = UUID.randomUUID().toString(); user.setPasswordResetToken(token);
user.setPasswordResetTokenExpiry(LocalDateTime.now().plusMinutes(15));
```
3. An email with `http://localhost:5173/reset-password?token=<uuid>` is sent.
4. On reset, the token is validated:

```
java if (user.getPasswordResetTokenExpiry().isBefore(LocalDateTime.now())) { throw new
IllegalArgumentException("Token has expired"); }
```
5. After reset, the token is **cleared** from the database so it cannot be reused.

Security note: The `forgotPassword` method silently succeeds even if the email is not found, preventing user enumeration attacks.

Q14: How does the admin disable a user account, and what safeguards exist?**Answer:**

Before disabling, the system calls the `order` service to check for active orders:

```
if (user instanceof Buyer) {
    if (orderClient.hasActiveOrdersForSpecificUser(user.getId(), "BUYER",
bearerToken)) {
        throw new IllegalStateException("Cannot disable: buyer has ongoing orders");
    }
} else if (user instanceof Seller) {
    if (orderClient.hasActiveOrdersForSpecificUser(user.getId(), "SELLER",
bearerToken)) {
        throw new IllegalStateException("Cannot disable: seller has ongoing
deliveries");
    }
}
user.setEnabled(false);
userRepository.save(user);
```

Once disabled, the `JwtFilter` also enforces the check at request time — even a user with a valid JWT cannot access the system if `user.isEnabled()` returns false:

```
if (user == null || !user.isEnabled()) {
    response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
    response.getWriter().write("{\"error\": \"Account is disabled\"}");
    return;
}
```

4. Order Lifecycle & Business Logic

Q15: Walk through the complete checkout flow.

Answer:

```

Frontend (CartPage) → POST /api/v1/orders/checkout
|
▼
OrderController.checkout()
  |— Extracts buyerId from JWT (authentication.getDetails())
  |— Extracts Bearer token for propagation
  |
  ▼
  OrderService.placeOrder()
    |
    |— Phase 1 – Validate: For each cart item,
    |   productClient.getProduct(productId) → calls product service
    |   Check: product exists AND stock >= requested quantity
    |   Throws 409 CONFLICT if insufficient stock
    |
    |— Phase 2 – Deduct stock:
    |   productClient.deductStock(productId, quantity, bearerToken)
    |   Passes JWT so product service accepts the authenticated call
    |
    |— Phase 3 – Persist order:
    |   Creates Order entity with status=PENDING, paymentStatus=UNPAID
    |   Creates OrderItem(s) with denormalized product snapshot
    |   Saves to bagify_orders database
    |   Returns OrderResponse to frontend
  
```

Q16: How does the Order status model work? What is the "hybrid" fulfillment model?

Answer:

There are **two levels** of status:

Per-item status (`OrderItem.ItemStatus`) — controlled by each seller:

PENDING → PROCESSING → PACKED → SHIPPED

Only admins can set `DELIVERED`.

Parent order status (`Order.OrderStatus`) — automatically **derived** from items:


```
public void computeStatus() {
    if (delivered == total)           → DELIVERED
    if (shipped + delivered == total) → SHIPPED
    if (shipped > 0 || delivered > 0) → PARTIALLY_SHIPPED
    if (processing > 0)               → PROCESSING
    else                             → PENDING
}
```

`computeStatus()` is called every time any item's status changes, and the parent order is re-saved. This means a multi-seller order automatically shows `PARTIALLY_SHIPPED` when one seller ships but another hasn't.

Q17: What are the conditions for a buyer to cancel an order?

Answer:

Only `PENDING` orders can be cancelled by the buyer:

```
if (order.getStatus() != Order.OrderStatus.PENDING) {
    throw new ResponseStatusException(HttpStatus.BAD_REQUEST,
        "Only PENDING orders can be cancelled");
}
```

After cancellation, **stock is restored** for every item:

```
for (OrderItem item : order.getItems()) {
    productClient.restoreStock(item.getProductId(), item.getQuantity(),
        bearerToken);
}
```

5. Concurrency & Transaction Management

Q18: How do you prevent duplicate payment processing from the webhook?

Answer:

Two layers of protection:

Layer 1 — Pessimistic Write Lock:

When a webhook arrives, the order row is immediately locked at the database level:

```
// OrderRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT o FROM Order o WHERE o.id = :id")
Optional<Order> findByIdForUpdate(@Param("id") Long id);
```

This executes `SELECT ... FOR UPDATE` in PostgreSQL. A second concurrent thread calling the same method blocks until the first transaction commits.

Layer 2 — Idempotency Flag:

Once the lock is acquired, the first thing checked is the current payment state:

```
boolean isAlreadyPaid = "PAID".equals(order.getPaymentStatus());
if (isAlreadyPaid) {
    log.info("Duplicate webhook: Order {} already PAID. Bypassing.", orderId);
    return; // exit without saving or calling downstream services
}
```

This ensures that even if a second thread acquired the lock after the first committed, it still does nothing.

Q19: Why is the seller stats update call wrapped in a try-catch inside the webhook handler?

Answer:

Marking the payment as `PAID` is the **critical operation**. Updating seller statistics is a **best-effort** secondary operation.

```
// After marking order as PAID and saving:
try {
    userClient.updateSellerStats(sellerId, revenue, quantity);
} catch (Exception e) {
    log.error("Could not update seller stats: {}", e.getMessage());
    // Order remains PAID – exception is swallowed intentionally
}
```

Why: If the `user` service is temporarily down, propagating the exception would roll back the entire `@Transactional` method, reverting `paymentStatus` back to `UNPAID` — even though the buyer already paid. This would be a critical data inconsistency. Catching the exception ensures the payment record is preserved.

Q20: Are there any race conditions in the checkout (stock deduction) flow?

Answer:

Yes, this is a recognised limitation. The checkout is a two-step process:

1. **Validate** stock $\geq$ quantity (read)
2. **Deduct** stock (write)

Between steps 1 and 2, another concurrent request could also pass validation and then both deduct stock, potentially going below zero. This is the classic **TOCTOU (Time-of-Check to Time-of-Use)** race condition.

Mitigation in the product service: The deduct-stock query is atomic:

```
UPDATE products SET stock = stock - :qty WHERE id = :id AND stock >= :qty
```

If the `WHERE stock >= :qty` condition fails (because another transaction already deducted), the update affects 0 rows and an exception is thrown.

6. Payment Gateway Integration (PayHere)

Q21: How do you prevent a user from tampering with the price before redirecting to PayHere?

Answer:

The backend generates a **signed hash** from the order details using the merchant secret (which the frontend never has access to):

```
hash = MD5( merchantId + orderId + amount + currency + MD5(merchantSecret) )
```

This hash is returned to the frontend and submitted to PayHere. When the redirect happens, PayHere's server **independently recalculates** this hash. If the user changed the amount in the browser, the hashes won't match and PayHere rejects the transaction.

Q22: How do you validate the authenticity of the server-to-server webhook notification?

Answer:

PayHere's server POSTs to `/api/v1/orders/payment/notify` with a `md5sig` parameter. The service validates it:

1. **Parameter presence check** — all required fields must be present.

2. **Signature recalculation:**

```
expectedSig = MD5( merchantId + orderId + payhere_amount + payhere_currency +
status_code + MD5(merchantSecret) )
```

3. **Case-insensitive comparison:**

```
java if (!calculatedMd5sig.equalsIgnoreCase(receivedMd5sig)) { throw new
ResponseStatusException(HttpStatus.BAD_REQUEST, "Invalid signature"); }
```

4. **Order state checks** — fetch order (with pessimistic lock), verify not cancelled, verify not already paid.

Q23: What are the possible PayHere status codes and how does your system handle them?

Answer:

status_code	Meaning	System Action
2	Payment successful	Set <code>paymentStatus=PAID</code> , <code>orderStatus=PROCESSING</code> , notify seller
0	Payment pending	Set <code>paymentStatus=PENDING</code> , no further action
-1	Payment cancelled	Set <code>paymentStatus=FAILED</code> , log warning
-2	Payment failed	Set <code>paymentStatus=FAILED</code> , log warning
-3	Chargebacked	Set <code>paymentStatus=FAILED</code> , log warning

The webhook endpoint is publicly accessible (no JWT required) because it is called by PayHere's server, which cannot provide a user JWT. Security is enforced entirely through signature verification.

7. Inter-Service Communication

Q24: How does the order service call the product service? What happens if the product service is down?

Answer:

The `order` service uses a `WebClient` (reactive HTTP client) to call the `product` service. The call is made via the Eureka logical name `lb://product`.

Token propagation — the buyer's JWT is forwarded in the `Authorization` header so the product service accepts the authenticated stock-deduction call.

If the product service is down: The `WebClient` call throws an exception. Since the checkout method is `@Transactional`, the transaction rolls back — no order is persisted and the user receives an error. There is no circuit breaker currently configured (a noted improvement would be adding Resilience4j).

Q25: How does the user service expose data to other services without breaking encapsulation?

Answer:

Dedicated **inter-service endpoints** are added to the `user` service:

```
// For product service – get seller name/rating when creating a product listing
GET /api/v1/users/sellers/{id} → returns SellerInfoResponse (id, name,
businessName, rating)

// For order service – get buyer shipping address at checkout
GET /api/v1/users/buyers/{id} → returns BuyerInfoResponse (id, name, email,
address, phone)
```

These return minimal DTOs (not the full User entity with password hash etc.), following the **principle of least privilege** for inter-service communication.

8. Notification Service

Q26: How does the notification service receive events? Why not use a message queue?

Answer:

Currently, the `notification` service receives **synchronous REST calls** from `order` and `user` services. For example, when a seller is verified, the `user` service calls:

```
POST /internal/notify/verification-result
```

The `/internal/**` routes are intentionally **not registered** in the API Gateway, so they are unreachable from the internet — only other backend services on the same network can call them.

Why not Kafka/RabbitMQ?

For this project scope, REST is appropriate. The code comment explicitly acknowledges: *"In production you'd swap to Kafka/RabbitMQ events — but REST is fine for now."* A message queue would add operational complexity (managing a broker) that is not justified for a university project.

Note: Despite RabbitMQ appearing in the `user` service `application.yml`, it is not actively used in the application logic.

Q27: What real-time notification mechanism is implemented?

Answer:

Server-Sent Events (SSE) via `SseService`. When an admin approves or rejects a seller, the seller receives an instant in-browser notification without polling:

```
// After saving the verification decision:
sseService.sendVerificationUpdate(sellerId,
updatedSeller.getVerificationStatus().name());
```

The seller's browser has an open SSE connection and receives the update pushed from the server in real-time.

9. Frontend Architecture

Q28: How is global state managed in the React frontend?

Answer:

React's **Context API** is used — no Redux. Four providers wrap the app in `App.tsx`:

```
<AuthProvider>          // JWT, user profile, role – persisted to localStorage
  <ProductProvider>      // product catalog, search, filters
    <WishlistProvider>    // bookmarked products
      <CartProvider>      // cart items, subtotal, checkout action
        <AppRoutes />
      </CartProvider>
    </WishlistProvider>
  </ProductProvider>
</AuthProvider>
```

Each context exposes state and action functions (e.g., `addToCart`, `removeFromWishlist`) via a custom hook (e.g., `useCart()`), preventing prop drilling.

Q29: How are routes protected from unauthorised access in React?

Answer:

A custom `ProtectedRoute` component wraps secure routes in `AppRoutes.tsx`:

```
<Route
  path="/admin-dashboard"
  element={
    <ProtectedRoute allowedroles={["ADMIN"]}>
      <AdminDashboard />
    </ProtectedRoute>
  }
/>
```

`ProtectedRoute` checks `AuthProvider`:

- Not authenticated → redirect to `/login`
- Wrong role → redirect to `/` (home)
- Correct role → render the child component

This prevents a buyer from manually typing `/admin-dashboard` in the URL and accessing the admin view.

Q30: Why was Vite chosen over Create React App?**Answer:**

Feature	Vite	Create React App
Dev server startup	Near-instant (native ESM)	Slow (webpack bundles everything)
HMR (Hot Module Replacement)	Instant, module-level	Slow on large projects
Production bundler	Rollup (smaller output)	Webpack
TypeScript support	Built-in	Requires config
Configuration	Minimal, extensible	Opinionated, harder to customise

10. Testing Strategy**Q31: What testing approach was used for the payment webhook?****Answer:**

Unit tests using **JUnit 5 + Mockito** in `OrderServiceWebhookTest.java`. The test class mocks all dependencies:

```
@ExtendWith(MockitoExtension.class)
class OrderServiceWebhookTest {
    @Mock private OrderRepository orderRepository;
    @Mock private OrderItemRepository orderItemRepository;
    @Mock private ProductClient productClient;
    @Mock private UserClient userClient;
    @InjectMocks private OrderService orderService;
}
```

Edge cases tested:

Test	Purpose
<code>testProcessPaymentNotification_Success</code>	Happy path — status code 2, order transitions to PROCESSING
<code>testProcessPaymentNotification_Pending</code>	Status code 0 — paymentStatus set to PENDING
<code>testProcessPaymentNotification_TamperedAmount</code>	Altered amount → signature mismatch → 400 error
<code>testProcessPaymentNotification_TamperedStatusCode</code>	Altered status code → signature mismatch → 400 error
<code>testProcessPaymentNotification_ArbitrarySignature</code>	Completely fake signature → 400 error
<code>testProcessPaymentNotification_MissingParameters</code>	Missing required field → 400 error
<code>testProcessPaymentNotification_NonExistentOrder</code>	Order ID not in DB → 404 error
<code>testProcessPaymentNotification_InactiveCancelledOrder</code>	Cannot pay for cancelled order → 400 error
<code>testProcessPaymentNotification_DuplicatePaidWebhook_SuccessStatus</code>	Already PAID → no save, no userClient call
<code>testProcessPaymentNotification_SellerStatsSyncFailure</code>	userClient throws exception → order still PAID, no rollback

Q32: Why use `ReflectionTestUtils` in the tests?**Answer:**

`@Value`-injected fields (like `merchantSecret`, `merchantId`, `currency`) cannot be injected by Spring in a pure Mockito unit test (no Spring context is loaded). `ReflectionTestUtils.setField` bypasses normal access controls to set private fields directly:

```
@BeforeEach
void setUp() {
    ReflectionTestUtils.setField(orderService, "merchantId", "1235988");
    ReflectionTestUtils.setField(orderService, "merchantSecret", "MzI20TA...");
    ReflectionTestUtils.setField(orderService, "currency", "LKR");
}
```

11. Potential Weaknesses & Improvements

Q33: What are the main weaknesses in the current implementation?**Answer:**

Area	Current State	Improvement
TOCTOU Race Condition	Stock check and deduction are two steps	Atomic SQL <code>UPDATE ... WHERE stock >= qty</code> in product service
Seller stats consistency	Stats updated via try-catch; a failure leaves them stale	Use an outbox pattern or event queue (RabbitMQ/Kafka)
No circuit breaker	If product service is down, checkout fails with an unhandled exception	Add Resilience4j <code>@CircuitBreaker</code> on <code>ProductClient</code>
Notification via REST	Synchronous REST calls to notification service add latency to the critical path	Move to async messaging (RabbitMQ/Kafka)
Hardcoded credentials	<code>application.yml</code> contains plain-text DB password and merchant secret	Use Spring Vault or environment variable injection
No rate limiting	Any client can hammer the webhook or auth endpoints	Add Spring Cloud Gateway rate limiter filter
No integration tests	Only unit tests for webhook; no tests for checkout flow	Add <code>@SpringBootTest</code> integration tests with Testcontainers

Q34: If you were to add a message queue (e.g., RabbitMQ), where would you use it?**Answer:**

The highest-value integration points would be:

1. **Order placed** → **Notification service** — instead of `order` service calling `POST /internal/notify/order-placed` synchronously, it publishes an `OrderPlacedEvent` message. The `notification` service consumes it asynchronously, sending the confirmation email without blocking the checkout response.
2. **Payment confirmed** → **Seller stats update** — instead of the fragile try-catch in `processPaymentNotification`, publish a `PaymentConfirmedEvent`. The `user` service consumes it and updates seller stats. If it fails, the message stays in the queue and is retried automatically.
3. **Seller verification decision** → **SSE** — instead of a direct `sseService.sendVerificationUpdate()` call, publish an event that the SSE handler consumes. This decouples the admin action from the real-time push.

This document was auto-generated based on a full analysis of the Bagify codebase including

`OrderController.java`, `OrderService.java`, `UserService.java`, `JwtFilter.java`, `SecurityConfig.java`, `OrderServiceWebhookTest.java`, and the API Gateway `application.yml`.